

Cours de Python

« Serpent scientifique et littéraire »



Programmation orientée objet en Python

- Chapitre1 :
- Présentation du langage Python

Présentation de python 1/3

- Python est un langage de programmation interprété, orienté objet et de haut niveau. Il a été créé en 1991 par Guido van Rossum et est devenu l'un des langages de programmation les plus populaires au monde en raison de sa simplicité, de sa clarté et de sa polyvalence.
- Python est utilisé dans de nombreux domaines, notamment la science des données, l'IA, le développement web, la finance et bien d'autres encore. Il est souvent considéré comme un langage de choix pour les débutants, mais il est également utilisé par les programmeurs expérimentés pour sa rapidité de développement et sa flexibilité.

Présentation de python 2/3

- Python est un langage de haut niveau, ce qui signifie qu'il gère les tâches de bas niveau telles que l'allocation de mémoire automatiquement. Il prend également en charge la programmation orientée objet, ce qui permet aux développeurs de créer des programmes modulaires et réutilisables.
- Python se distingue des autres langages de programmation par sa syntaxe concise et lisible, ce qui facilite l'apprentissage et la compréhension du code. De plus, il est doté d'une vaste bibliothèque standard qui contient de nombreuses fonctionnalités pour résoudre divers problèmes de programmation.

Présentation de python 3/3

- En outre, Python est multiplateforme, ce qui signifie qu'il peut être exécuté sur différents systèmes d'exploitation tels que Windows, Linux et MacOS. Python possède également une grande communauté de développeurs qui travaillent constamment à améliorer le langage et à créer de nouvelles bibliothèques.

Installation

❖ Sous Windows

- Téléchargez la dernière version du programme d'installation de Python depuis le site web officiel de Python

<https://www.python.org/ftp/python/3.11.3/python-3.11.3-amd64.exe>

- Ajouter Python aux variable d'environnement « PATH »

Installation

❖ Sous Linux

- Vérifiez si Python est déjà installé en ouvrant un terminal et en tapant "python" ou "python3". Si Python est déjà installé, il affichera la version installée. Si ce n'est pas le cas, continuez avec l'étape suivante.
- Ouvrez un terminal et tapez "**sudo apt-get update**" pour mettre à jour les packages.
- Tapez "**sudo apt-get install python3**" pour installer Python 3. Si vous avez besoin d'installer une version spécifique de Python, remplacez "**python3**" par la version que vous souhaitez installer.
- Une fois l'installation terminée, vous pouvez ouvrir l'interpréteur Python en tapant "python" ou "python3" dans un terminal.

Installation

❖ Sous MacOS

- Téléchargez le programme d'installation de Python depuis le site web officiel de Python

<https://www.python.org/downloads/macos/>

- Ouvrez le programme d'installation et suivez les instructions à l'écran pour installer Python.

Programmation orientée objet en Python

- Chapitre 2:
- Syntaxe, Variables et Types de données

Les commentaires

- Les commentaires ne sont pas exécutés par Python, ils sont destinés aux humains et non à l'ordinateur
- Ils facilitent la lecture et la compréhension du programme par d'autres personnes (ou par soi-même quelques années plus tard !)
- Commentaires simples (mono lignes)
 - Les commentaires commencent par un dièse #
 - Ils se prolongent jusqu'à la fin de la ligne
- # Ceci est mon premier commentaire !
- Commentaires longs
 - Il n'y a pas de commentaire long (type /* */) à proprement parler en Python
 - ✗ Commenter chaque ligne
 - ✗ Utiliser une chaîne de caractères longue :
- ```
"""Ceci est une chaîne longue non utilisée,
qui fait office de commentaire multi-ligne !
"""
```

# Afficher à l'écran

- La fonction `print()` permet d'écrire à l'écran (dans le terminal)

```
>>> print(5 + 2)
7
>>> print("Bonjour")
Bonjour
>>> print("Bonjour", "Master SI/SR")
Bonjour Master SI/SR
```

- En ligne de commande, le `print()` peut être omis

```
>>> 5 + 2
7
```

# Aide

## ■ Plusieurs commandes permettent d'obtenir de l'aide en Python

- Aide sur une classe ou une méthode :

```
help(list)
help(list.append)
```

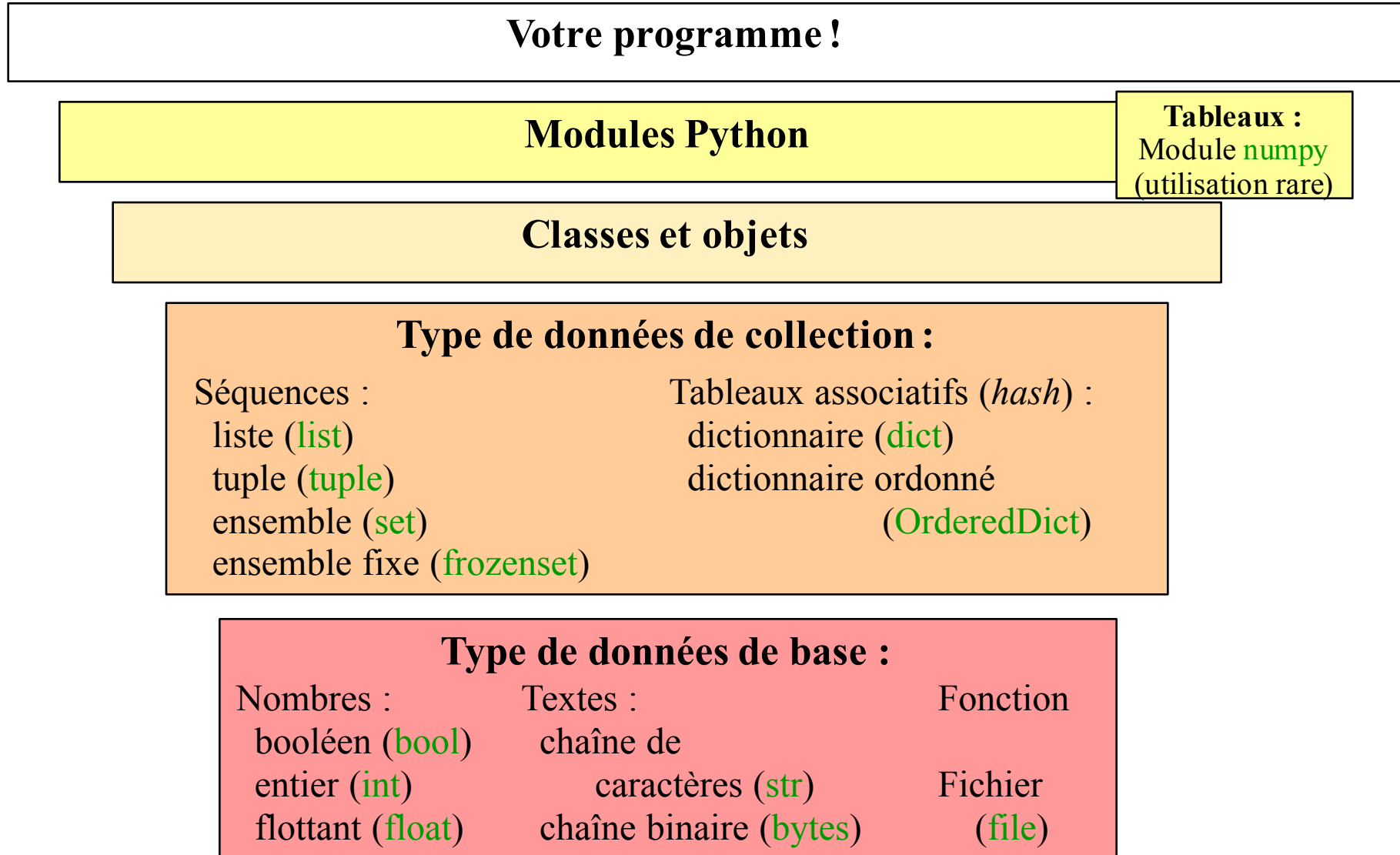
- Lister les méthodes d'un objet :

```
dir(objet)
```

## ■ Aide en ligne

- <https://docs.python.org/3/>
- Aussi disponible en téléchargement ou en paquet Linux

# Les types de données



# Nombres entiers

- ```
>>> print(1)
1
>>> print(1 + 2)
3
>>> print((3 + 4) * (5 + 8))
91
```
- Pas de limite aux nombres entiers en Python !
- ```
>>> print(123456789123456789123456789)
123456789123456789123456789
```

# Nombres à virgule

- `>>> print(1.3)`

1.3

- `>>> print(1.3e6)`

1300000.0

- La précision du flottant Python correspond en fait à un double.

# Booléens

■ Deux valeurs possibles : vrai ou faux

× **True**

× **False**

# Variables

- Une variable est un nom auquel on associe une valeur
  - ✗ La valeur est souvent connue seulement à l'exécution du programme (par exemple c'est le résultat d'un calcul)
- Sous Python, les noms de variables doivent en outre obéir à quelques règles simples :
  - ✗ Un nom de variable est une séquence de lettres ( $a \rightarrow z, A \rightarrow Z$ ) et de chiffres ( $0 \rightarrow 9$ ), qui doit toujours commencer par une lettre.
  - ✗ Seules les lettres ordinaires sont autorisées. Les lettres accentuées, les cédilles, les espaces, les caractères spéciaux tels que \$, #, @, etc. sont interdits, à l'exception du caractère `_` (souligné).
  - ✗ Attention majuscules et minuscules sont différenciées !
  - ✗ Il est important de trouver des noms parlants

# Variables

■ En plus de ces règles, il faut encore ajouter que vous ne pouvez pas utiliser comme nom de variables les 33 « mots réservés » ci-dessous

|      |          |        |        |       |          |        |
|------|----------|--------|--------|-------|----------|--------|
| and  | as       | assert | break  | class | continue | def    |
| del  | elif     | else   | except | False | finally  | for    |
| from | global   | if     | import | in    | is       | lambda |
| None | nonlocal | not    | or     | pass  | raise    | return |
| True | try      | while  | with   | yield |          |        |

# Affectations simples de variables

■ On crée une variable en lui donnant une valeur, avec =

```
>>> age = 36
```

■ On obtient la valeur d'une variable

- En donnant son nom

```
>>> age
```

```
36
```

Ou bien avec **print**

```
>>> print(age)
```

```
36
```

■ La valeur de la variable peut être modifiée avec =

```
>>> age = 37
```

```
>>> age = age + 1 # Anniversaire
```

```
37
```

■ **del** permet de supprimer une variable. **del age**

# Affectations multiples

- Sous Python, on peut assigner une valeur à plusieurs variables simultanément

```
>>> x = y = 2020
[>>> x
2020
[>>> y
2020
>>> █
```

- On peut aussi effectuer des affectations parallèles à l'aide d'un seul opérateur :

- ```
[>>> note1, note2 = 13, 18.77
[>>> note1
13
[>>> note2
18.77
>>> █
```



Types de données

■ Python peut manipuler différents types de données :

✗ des nombres entiers (integer en anglais, abrégé en int),

✗ des nombres à virgule (souvent appelé flottants ou float en anglais),

✗ des chaînes de caractères (string en anglais, abrégé en str)

✗ et des booléens (valeur vraie ou fausse, bool) :

- age = 25 # Entier
- Poids = 35.5 # Flottant
- nom = "Dahirou Gueye" # Chaîne de caractères
- enseignant = TRUE # Booléen
- etudiant = FALSE # Booléen
- telephone = "01 58 38 85 78" # Chaîne de caractères!

Opérations sur les nombres

■ Opérations basique :

✗ **Addition** : +

✗ **Soustraction** : -

✗ **Multiplication** : *

✗ **Division** : /

✗ **Division entière** : //

✗ **Modulo (reste de la division)** : %

✗ **Puissance** : **

■ D'autres opérations sont définies dans le module **math**

Priorité des opérations

■ Sous Python, les règles de priorité sont les mêmes que celles qui vous ont été enseignées au cours de mathématique. Vous pouvez les mémoriser à l'aide de l'acronyme PEMDAS :

- P pour parenthèses
- E pour exposants
- M et D pour multiplication et division, qui ont la même priorité
- A et S l'addition A et la soustraction S

Chaînes de caractères

- Les chaînes de caractère s'écrivent entre guillemets (simples ou doubles)
 - > `Structure="Institut de Formation à Distance"`
 - > `chaine_vide=" "`
- Les chaînes de caractère sont en Unicode
 - Cela permet de gérer toutes les langues du monde et des symboles
- Pas de type caractère en Python
 - On utilise une chaîne d'un seul caractère
- Caractères spéciaux
 - Retour à la ligne : `"\n"`
 - Tabulation : `"\t"`
 - Antislash : `"\\"`

Chaînes de caractères

■ Chaînes avec des guillemets à l'intérieur

- « Antislasher » les guillemets à l'intérieur de la chaîne : **"Il se nomme \"Dahirou\"."**
- Tripler les guillemets extérieurs : **"""Il se nomme "Dahirou".""""**
- Alternier guillemets doubles et simples : **'Il se nomme "Dahirou".'**

Chaînes de caractères

■ Opérations sur les chaînes

- ✗ Obtenir la longueur d'une chaîne (= le nombre de caractères)
- ✗ Obtenir un caractère de la chaîne
- ✗ Obtenir une partie de la chaîne
- ✗ Concaténer deux chaînes
- ✗ Rechercher si une chaîne est incluse dans une autre

• Exemples avec `s = "Bonjour"`

`len(s)` $\Rightarrow$ 7

`s[0]` $\Rightarrow$ "B" `s[-1]` $\Rightarrow$ "r"

`s[0:3]` $\Rightarrow$ "Bon"

`"Hi " + s`

`s.find("jour")` $\Rightarrow$ 3 # Trouvé en position 3
(-1 si pas trouvé)

Chaînes de caractères

■ Opérations sur les chaînes

✗ Passer une chaîne en minuscule ou en majuscule

• "

`s.lower()`

`s.upper()`

✗ Remplacer un morceau d'une chaîne

`s.replace("Bonjour", "Good morning")`

✗ Demander à l'utilisateur de saisir une chaîne

`saisie = input("Entrez un nom : ")`

Chaînes de caractères

- Formatage de chaînes de caractère : inclure des variables dans une chaîne : %

```
>>> nom = "Gueye »
```

```
>>> prenom = "Dahirou"
```

```
>>> "Salut %s !" % prenom  
Salut Dahirou !
```

```
>>> "Bonjour %s %s !" % (prenom, nom)
```

```
>>> "Taux de réussite : %s %%" % 100
```

```
Taux de réussite : 100 %
```

- Attention aux faux nombres !

```
telephone = "01 48 38 73 34"
```

- Quel est le type de donnée de la variable ci-dessus ?
- Pourquoi a-t-on choisi ce type de donnée ?

Chaînes de caractères

■ Attention aux faux nombres !

telephone = "01 58 45 77 89"

✗ Quel est le type de donnée de la variable ci-dessus ?

✗ Chaîne de caractères

✗ Pourquoi a-t-on choisi ce type de données ?

✗ À cause du 0 initial

✗ Les chiffres sur lesquels on ne fait pas de calcul ne sont pas des nombres mais des chaînes de caractères !

Conversions

- Il est souvent nécessaire de convertir d'un type de données vers un autre
 - Les fonctions `int()`, `float()`, `bool()` et `str()` permettent de convertir une valeur vers un entier, un flottant et une chaîne de caractère
 - `int("8")` $\Rightarrow$ 8
 `str(8)` $\Rightarrow$ "8"
 - `input()` retourne toujours une chaîne de caractères ; il faut penser à la transformer en entier ou en flottant si l'on demande la saisie d'un nombre !

```
age = int(input("Entrez votre âge : "))
```

```
poids = float(input("Entrez votre poids : "))
```